

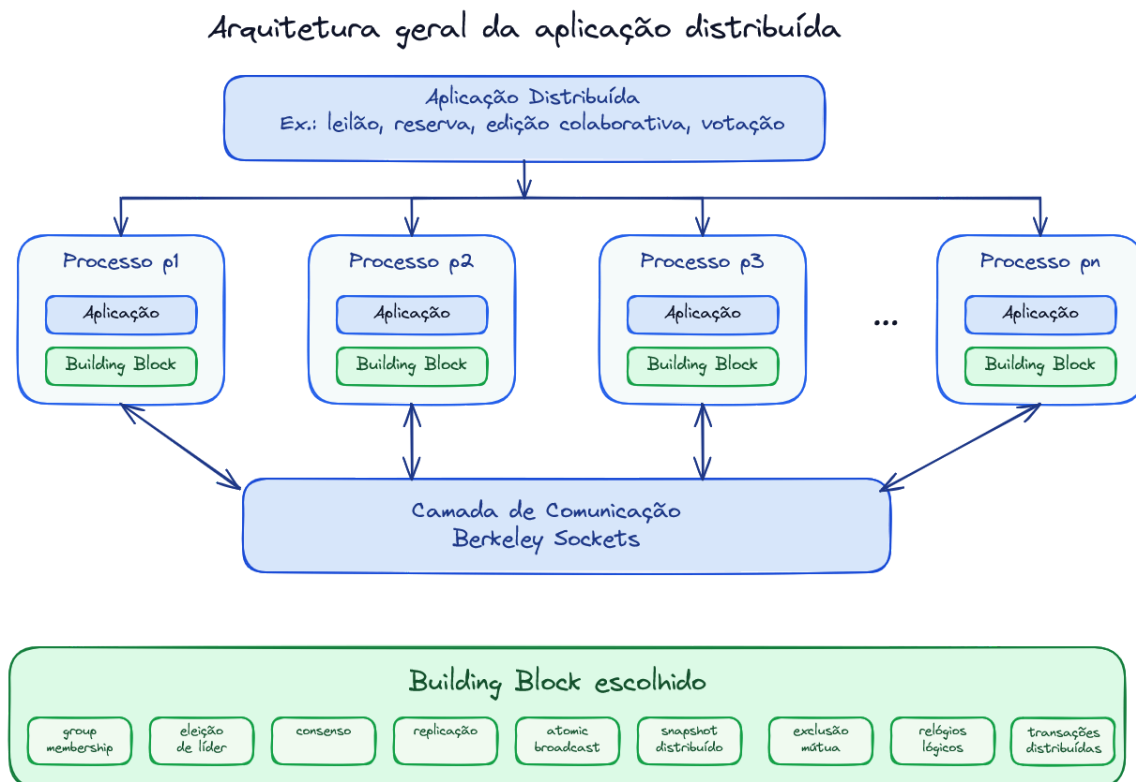
Definição do Trabalho 2: Aplicação Distribuída baseada em Building Blocks

Sistemas distribuídos são compostos por processos independentes que se comunicam por troca de mensagens e precisam coordenar suas ações mesmo na presença de concorrência, atrasos de comunicação e falhas parciais. Para isso, diferentes mecanismos fundamentais podem ser utilizados, como eleição de líder, consenso, replicação, difusão atômica, exclusão mútua distribuída, snapshots distribuídos, relógios lógicos e transações distribuídas.

Neste trabalho, deve ser implementada uma aplicação distribuída que utilize, de forma central, um ou mais conceitos fundamentais de Computação Distribuída. O objetivo é que o grupo implemente um algoritmo para um dos mecanismos fundamentais e implemente uma aplicação de exemplo em que seja possível observar claramente o funcionamento do mecanismo implementado.

A aplicação deve ser composta por múltiplos processos distribuídos, que se comunicam entre si por meio de *sockets* (conformidade com *Berkeley sockets*).

Arquitetura do Sistema



Processos independentes se comunicam por uma camada de comunicação e utilizam um building block distribuído para coordenar ações, ordenar eventos, replicar estado ou observar o comportamento global do sistema.

A arquitetura geral do sistema deve conter, no mínimo, os seguintes elementos:

- **Processos distribuídos:** representam os nós participantes da aplicação.
- **Camada de comunicação:** responsável pela troca de mensagens entre os processos.
- **Building block distribuído:** conceito principal, será sorteado para cada grupo.
- **Aplicação final:** sistema funcional que utiliza o *building block* implementado para resolver um problema prático.

Cada grupo deve definir a arquitetura específica da sua aplicação, indicando quais processos existem, como eles se comunicam e qual papel cada um desempenha no sistema. A aplicação pode ser executada em uma única máquina, utilizando diferentes portas para representar diferentes processos.

Aplicação

O grupo deve implementar uma aplicação distribuída funcional. A aplicação não deve apenas executar um algoritmo isolado, mas sim utilizar o conceito escolhido como parte essencial do seu funcionamento.

Exemplos de aplicações possíveis (Obs. não se limite a esses exemplos):

- **Sistema de edição colaborativa simplificado:** usuários editam um documento compartilhado e o sistema precisa ordenar ou reconciliar eventos. Pode usar relógios lógicos, ordem causal, replicação ou *atomic broadcast*.
- **Leilão distribuído:** vários participantes enviam lances ao mesmo tempo, e o sistema precisa decidir o vencedor de forma consistente. Pode usar consenso, *atomic broadcast* ou eleição de líder.
- **Sistema de reserva de salas ou assentos:** múltiplos usuários tentam reservar o mesmo recurso ao mesmo tempo. Pode usar exclusão mútua distribuída, consenso, replicação ou transações distribuídas.
- **Aplicação de votação com participantes maliciosos:** alguns nós podem enviar votos ou mensagens inconsistentes para tentar alterar o resultado. Pode usar consenso e/ou replicação bizantina.

Temas / Building Blocks

Cada grupo terá sorteado **um building block principal** para ser o foco do trabalho. Opcionalmente, o grupo pode utilizar um segundo conceito como complemento.

Os temas possíveis para *building blocks* são:

- **Group membership / view synchrony:** manter uma visão dos processos participantes do grupo, permitindo observar entrada, saída ou falha de processos, com a atualização consistente da visão do grupo.
- **Eleição de líder:** permitir que os processos escolham um líder responsável por coordenar alguma parte da aplicação, como ordenação de mensagens, distribuição de tarefas, tomada de decisão ou controle de acesso a recursos.

- **Consenso:** permitir que múltiplos processos cheguem a uma mesma decisão, mesmo partindo de propostas diferentes.
- **Replicação:** manter cópias de um mesmo estado em múltiplos processos, demonstrando como as atualizações são propagadas e qual modelo de consistência foi adotado.
- **Atomic broadcast:** permitir que mensagens sejam difundidas para múltiplos processos, garantindo a entrega e a mesma ordem de entrega por todos os participantes corretos.
- **Consenso e/ou replicação bizantina:** implementar uma solução de consenso e/ou replicação considerando a possibilidade de processos com comportamento bizantino, isto é, processos que podem enviar mensagens incorretas, conflitantes ou inconsistentes para diferentes participantes.
- **Snapshot distribuído:** capturar um estado global consistente de uma aplicação distribuída.
- **Exclusão mútua distribuída:** controlar o acesso concorrente a um recurso compartilhado, garantindo que apenas um processo acesse a seção crítica por vez.
- **Relógios lógicos e ordem causal:** usar relógios lógicos, como relógios de Lamport ou relógios vetoriais, para ordenar eventos ou identificar relações de causalidade.
- **Transações distribuídas:** permite organizar operações em transações distribuídas, de forma que um protocolo de controle de concorrência faz a validação das transações concorrentes. Por exemplo, o protocolo garante que transações concorrentes T1 e T2, em que pelo menos uma operação de T1 conflita com uma operação de T2, apenas uma transação (T1 ou T2) efetivará (a outra será abortada).

Observe que para cada conceito representado por um *building block*, há inúmeros algoritmos disponíveis, desde os clássicos até o estado da arte. Por exemplo, para consenso, há algoritmos como Paxos e suas variações (Multi-Paxos, Flexible Paxos, Egalitarian Paxos, etc.), Raft (bastante popular, usado pelo key-value store etcd), Zab (usado pelo Zookeeper) e Viewstamped Replication. De forma análoga, há vários algoritmos para eleição de líder (Bully, Chang Roberts, Peterson, *randomized*, etc.) ou para qualquer dos conceitos apresentados.

O grupo deve escolher e estudar o algoritmo utilizado na implementação do seu trabalho.

Requisitos

O trabalho deve atender aos seguintes requisitos:

- Desenvolvimento:
 - implementar uma aplicação distribuída funcional;
 - utilizar pelo menos três processos independentes;
 - implementar pelo menos um *building block* principal;
 - utilizar comunicação real entre processos (*Berkeley Sockets*);
- Demonstração (gravação de uma demo: máximo de 10 minutos):
 - apresentar logs ou saídas que permitam observar o funcionamento do sistema;
 - demonstrar um cenário normal de execução;

- demonstrar um cenário com concorrência, falha, atraso ou comportamento especial relacionado ao tema escolhido;
- Seminário (apresentação em aula: 10 a 12 minutos)
 - Discutir as principais decisões de implementação
 - Explicar o algoritmo e API disponibilizada por seu *building block*
 - Ilustrar o uso do *building block* com a sua aplicação de exemplo

Recomendações para os seminários e demonstrações:

Considere que os seus colegas não conhecem os detalhes sobre o conceito, algoritmo e aplicação desenvolvida. Então procure evidenciar detalhes que sejam importantes para a compreensão. Por exemplo:

- em uma aplicação com replicação, deve ser possível observar o estado das réplicas ao final da execução;
- em uma aplicação com exclusão mútua, deve ser possível observar que apenas um processo entra na seção crítica por vez;
- em uma aplicação com *atomic broadcast*, deve ser possível observar que todos os processos entregam as mensagens na mesma ordem;
- em uma aplicação com snapshot distribuído, deve ser possível observar o estado global capturado;
- em uma aplicação com relógios lógicos, deve ser possível observar os *timestamps* atribuídos aos eventos.

Entrega

O trabalho consiste em:

1. Implementar a aplicação distribuída
2. Entregar o código-fonte
3. Entregar instruções claras de compilação e execução
4. Entregar os slides da apresentação
5. Enviar o link para acesso à demonstração gravada
6. Apresentar o trabalho em seminário

O trabalho pode ser realizado em grupos de **até 3 participantes**.

Todos os artefatos e link para vídeo devem ser enviados pelo Moodle para análise e avaliação. Os nomes dos participantes do grupo devem constar nos artefatos entregues. Participantes com nomes não referenciados não serão considerados como membros do grupo.

Apresentação

O trabalho será apresentado em formato de seminário, com duração entre **10 e 12 minutos** por grupo.

A apresentação deve conter, pelo menos:

- problema estudado;
- aplicação implementada;

- *building block* utilizado;
- arquitetura da solução;
- algoritmo implementado;
- cenário normal de execução;
- cenário com concorrência, falha, atraso ou comportamento especial;
- principais dificuldades;
- limitações e conclusões.

Importante: A participação nos seminários (perguntas e interações da platéia), faz parte da avaliação.